CASE STUDY REPORT | LOW-CODE TO PRO-CODE

Exploring Pro-Code Extension of a Zoho Creator Application

using Zoho Catalyst and Custom Functions

NAME	ROLL NUMBER	DEPARTMENT
Madhumitha P	**241401049**	**CSBS**

241401049

1 Abstract

Zoho Creator is a low-code platform for building business applications quickly. Its scripting language, Deluge, handles most day-to-day logic. But when an application needs heavy computation, third-party libraries, document processing, scheduled back-end jobs or custom APIs, Deluge alone is not enough. This report explores how a Creator application can be extended with **pro-code**, meaning real programming languages, using **Zoho Catalyst** serverless functions and Deluge custom functions.

The report explains the two platforms in simple terms, compares the available extension methods, presents a reference architecture, and walks through an implementation for an illustrative Expense Claim application. It closes with security considerations, limitations, best practices and a short guide for deciding when to stay in Deluge and when to move to Catalyst.

Key idea: Keep the **screens, forms and simple workflows** in Creator, and move **complex logic, heavy processing and integrations** into Catalyst functions that Creator calls over HTTPS.

2 Introduction

2.1 Why this topic matters

Low-code tools let teams build working apps in days. Over time, though, real business needs grow. A form that once collected simple data now has to validate tax rules, read a scanned receipt, create a formatted PDF and sync with a bank. These tasks need more power than a visual builder usually offers.

Instead of abandoning the platform, a better approach is to **extend** it. The Creator app stays as the user-facing layer, and code written in Java, Node.js or Python does the heavy work behind the scenes.

2.2 Objectives of the report

1. Understand what Zoho Creator, Deluge and Zoho Catalyst provide.
2. Identify the situations where Deluge reaches its limits.
3. Compare the ways to connect Creator with custom code.
4. Design and describe a working architecture with sample code.
5. Summarize risks, best practices and a decision guide.

2.3 Scope and method

The report is based on the public documentation of Zoho Catalyst and Zoho Deluge, plus a worked design example. The Expense Claim application is illustrative. Code snippets show the pattern and must be adapted to your own project names, URLs and authentication settings.

241401049

3 Background: Zoho Creator and Deluge

3.1 What Zoho Creator offers

Zoho Creator lets you build apps with forms, reports, pages, dashboards and workflows through a visual interface. It handles hosting, user management, permissions and mobile access for you. Business analysts can build much of an application without writing code.

3.2 Deluge: the built-in scripting language

Deluge is Zoho's own English-like scripting language. It runs on the server inside Creator and other Zoho products. Developers use it for form events (on submit, on edit), scheduled tasks, custom buttons and API calls. A key feature is the **invokeUrl** task, which acts as an HTTP client so that a script can call an outside service or another Zoho API.

A **custom function** in Creator is a reusable Deluge function that can be called from many workflows. It keeps logic in one place, which is good practice, but it still runs in the same Deluge environment with the same limits.

3.3 Where Deluge reaches its limits

Need	Why Deluge struggles
Heavy calculation or loops over large data	Deluge enforces execution ceilings, such as a cap on statements per function (commonly cited as 5,000) and a time limit per run
Third-party libraries	You cannot import packages such as PDF, image or machine-learning libraries
Complex data structures and algorithms	The language is simple by design and has limited data handling
Custom API endpoints	Creator is not meant to host your own REST services
Long-running or background jobs	Scheduled scripts are bound by the same execution limits
Unit testing and version control	Limited tooling compared with standard software projects

> **Note:** Zoho publishes the current limits in its documentation, and they can change by plan and release. Always check the latest values before designing around a number.

4 Zoho Catalyst Overview

4.1 What is Catalyst?

Zoho Catalyst is a full-stack, serverless development platform from Zoho. "Serverless" means you write the code and Catalyst runs it, scales it and manages the servers. You are billed for usage and do not maintain machines.

4.2 Catalyst Functions

Functions are the core building block. According to the Catalyst documentation, there are **five types** of server-side functions, and all five can be written in **Java, Node.js or Python**.

Function type	Purpose	Typical use with Creator
Basic I/O	Simple input and output with a string parameter	Small helper calls, quick calculations
Advanced I/O	Full HTTP functions with headers and native request and response objects	Main REST endpoints that Creator calls
Event	Triggered by events set up through event listeners	React to changes in Catalyst services
Cron	Runs once or on a schedule	Nightly reconciliation, report generation
Integration	Backends for other Zoho services	Zoho service integrations (see note below)

Catalyst also provides **Security Rules**, which control who can invoke Basic I/O and Advanced I/O functions, and an **API Gateway**, which handles routing, throttling and authentication of client requests.

> **Availability note:** The documentation states that Integration Functions are not available in the EU, AU, IN and CA data centers at the time of writing. For those regions, Advanced I/O functions with REST calls are the portable choice, and this report follows that approach.

4.3 Other useful Catalyst services

- **Data Store:** a relational database for data that should live outside Creator.
- **File Store:** storage for documents, images and generated files.
- **Cache:** fast storage for frequently used values.
- **Zia services:** ready-made AI features such as OCR, which can read text from images.
- **Cron and Event listeners:** scheduling and event-based triggering.
- **Client hosting:** a place to host a front-end if one is needed.

5 Ways to Extend a Creator Application

There is more than one way to add pro-code power. The right choice depends on how heavy the task is and where it must run.

Method	How it works	Strength	Limitation
A. Deluge custom function	Reusable script inside Creator	Easy, no extra platform	Same Deluge limits
B. Creator calls a Catalyst function	Deluge sends an HTTPS request with invokeUrl; function returns JSON	Full programming power, libraries, scale	Network latency, extra setup

Method	How it works	Strength	Limitation
C. Catalyst calls Creator	Cron or Event function uses Creator REST API to read and write records	Back-end jobs and bulk updates	Needs OAuth and API limits planning
D. Widgets and client scripts	HTML and JavaScript embedded in a Creator page	Custom user interface	Front-end only; not for secure logic
E. Webhooks and API	External systems and Creator exchange data through endpoints	Loose coupling	Needs careful security

Methods **B** and **C** are the core of a pro-code extension. Together they form a two-way link: Creator can ask Catalyst to do work, and Catalyst can push results or scheduled updates back into Creator.

5.1 How the Creator side is cost-aware

Each invokeUrl execution makes a real API call in the background. Zoho states that these calls count against the **external calls limit** of the Creator plan, and a call placed inside a loop is counted every time it runs. This means that calling a function once for 500 records is far better than calling it 500 times.

6 Reference Architecture

The diagram below shows how the pieces fit together. Users work only in Creator. Creator calls a secured Catalyst endpoint, which runs the function and, if needed, uses other Catalyst services. Results come back as JSON, and scheduled Catalyst jobs can update Creator through its REST API.

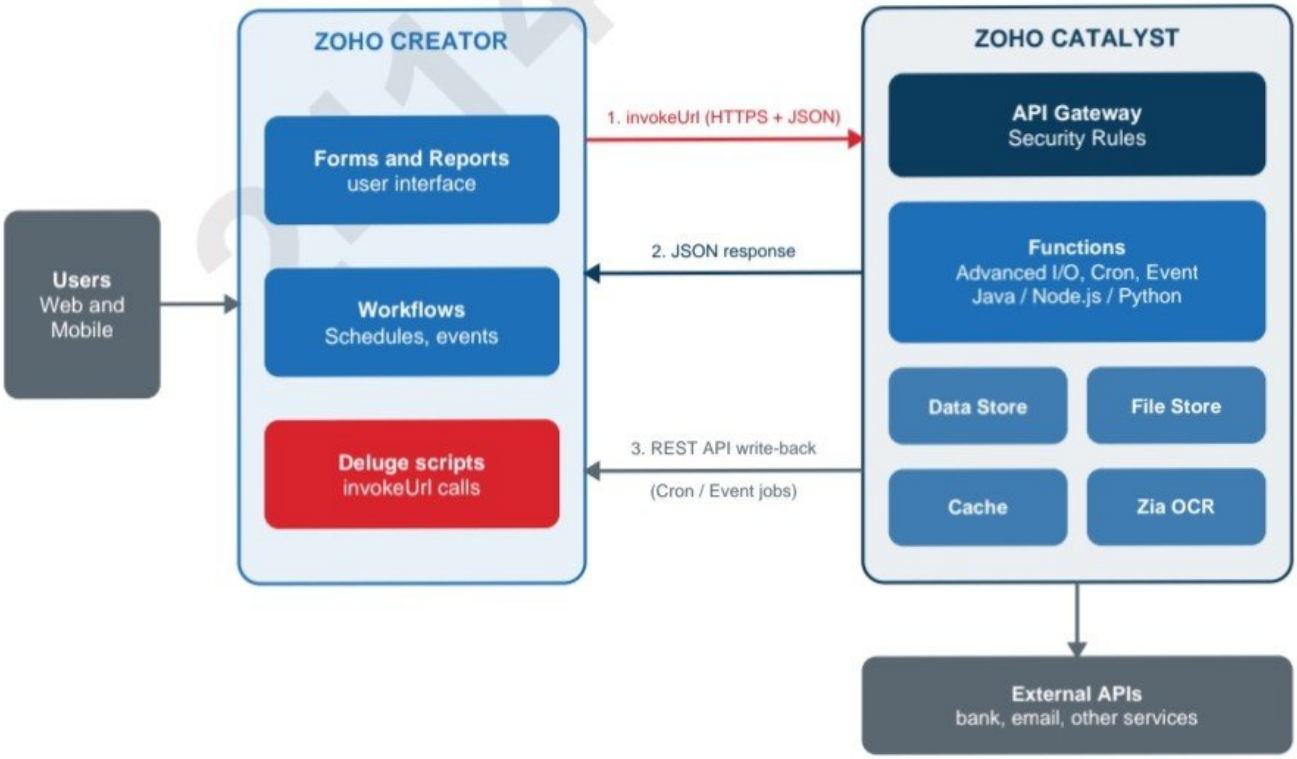


Figure 1: Reference architecture linking Zoho Creator with Zoho Catalyst

6.1 Request flow in plain steps

1. A user submits a form in the Creator app.
2. The on-submit Deluge script collects the needed fields and builds a JSON payload.
3. Deluge calls the Catalyst endpoint with **invokeUrl**, sending credentials through a connection or a secure header.
4. The API Gateway and Security Rules check the request.
5. The Advanced I/O function runs the business logic and may use Data Store, File Store or Zia.
6. The function returns a JSON response with a status and results.
7. Deluge reads the response, updates the record and notifies the user.

6.2 Design principle

Business rules that change often can still live in Creator if they are simple. Rules that are complex, shared with other systems or computation-heavy belong in the function, so that they can be tested and reused.

7 Implementation Example: Expense Claim App

7.1 The scenario

An organization uses a Creator app for employee expense claims. Employees upload a receipt and enter an amount. As the app grows, the finance team asks for four things that Deluge cannot handle comfortably.

Requirement	Why pro-code is needed	Catalyst piece
Check each claim against a policy with many rules by grade, city and category	Complex, testable logic	Advanced I/O function
Read amount and date from the receipt image	OCR and image handling	Zia OCR with a function
Generate a formatted monthly PDF report	Document libraries	Function plus File Store
Reconcile approved claims with the bank every night	Background job	Cron function

7.2 Step 1: Build the Catalyst function

After creating a Catalyst project and an Advanced I/O function (using the Catalyst CLI or console), the function receives JSON and returns a decision. The following Node.js sketch is illustrative.

```
const express = require("express");
const app = express();
app.use(express.json());
// Policy limits per category (kept in code or Data Store)
const LIMITS = { Travel: 5000, Meals: 1500, Lodging: 8000 };
app.post("/validate", (req, res) => {
  const { claimId, category, amount, grade } = req.body;
  const limit = (LIMITS[category] || 0) * (grade === "Senior" ? 1.2 : 1);
  if (!limit) {
    return res.status(400).json({ status: "error", reason: "Unknown category" });
  }
  const approved = amount <= limit;
  res.status(200).json({
    status: approved ? "Auto-Approved" : "Needs Review",
    claimId,
    limit,
    reason: approved ? "Within policy" : "Exceeds limit of " + limit
  });
});
module.exports = app;
```

7.3 Step 2: Secure the endpoint

Do not leave the function open to the internet. Use Catalyst Security Rules and, where suitable, the API Gateway to control access and set rate limits. Store secrets such as keys in environment variables, never in the code or in the Creator script.

7.4 Step 3: Create a connection in Creator

In Creator, set up a **Connection** so that credentials are stored centrally and are not written in scripts. Use OAuth where the target supports it. If you use a token or key in a header instead, keep it in a secure place and rotate it regularly.

7.5 Step 4: Call the function from Deluge

The on-submit workflow below builds the payload, calls the function, and updates the claim. It is a pattern, so URLs and names must be replaced with your own.

```
// Creator form workflow: On Success of Expense_Claim
payload = Map();
payload.put("claimId", input.ID.toString());
payload.put("category", input.Category);
payload.put("amount", input.Amount);
payload.put("grade", input.Employee.Grade);
response = invokeurl
[
  url :"https://<your-catalyst-domain>/server/claim_fn/validate"
  type :POST
  parameters :payload.toString()
  headers :{"Content-Type":"application/json"}
  connection :"catalyst_connection"
];
info response;
if(response.get("status") != null)
{
  input.Status = response.get("status");
  input.Remarks = response.get("reason");
}
```

```
else
{
  input.Status = "Error";
  input.Remarks = "Validation service unavailable";
}
```

7.6 Step 5: Background jobs and write-back

For the nightly bank reconciliation, a **Cron function** runs on schedule. It reads approved claims through the Creator REST API, matches them with bank data, and writes the matching status back to Creator. OAuth credentials with the minimum required scopes should be used, and API limits should be planned so that the job processes records in batches.

7.7 Step 6: Test and deploy

- Run the function locally with the Catalyst CLI and test with sample JSON.
- Deploy to the **development** environment first and test the full flow from Creator.
- Move to **production** only after the error cases have been tested.
- Keep the function code in a version control system such as Git.

8 Security, Reliability and Performance

8.1 Security checklist

- Authenticate every call. Use connections, OAuth or gateway authentication.
- Apply the principle of least privilege to OAuth scopes and roles.
- Validate all input inside the function. Never trust data only because it came from Creator.
- Keep secrets in environment variables and rotate them.
- Log requests without writing personal or financial data to logs.
- Respect data residency rules by choosing the right Catalyst data center.

8.2 Reliability

Network calls can fail. Deluge scripts should always check the response, handle missing values, and set a clear status such as "Error" when the service is not reachable. For important operations, design the function to be **idempotent**, so that a retry does not create a duplicate result.

8.3 Performance

- Send one request for many records instead of many requests for one record.
- Return only the fields Creator needs.
- Cache values that do not change often.
- Move long jobs to Cron or Event functions rather than waiting for them inside a form submit.

241401049

9 Benefits and Challenges

Benefits	Challenges
Removes Deluge's execution and library limits	Two platforms to learn, build and maintain
Logic can be tested, reused and versioned like normal software	Network latency on every call
Serverless: no servers to manage, scales with demand	Calls count toward Creator's external call limits
Creator remains the fast front end for business users	Debugging across two systems is harder
Supports AI, OCR, documents and custom APIs	Needs developer skills and good documentation
Gradual migration: move only what needs it	Costs of Catalyst usage must be monitored

10 Best Practices and Decision Guide

10.1 Best practices

1. **Start in Deluge,** and move logic out only when a real limit appears.
2. **Keep the contract clear.** Define the JSON request and response, and keep them stable.
3. **One purpose per function.** Small functions are easier to test and secure.
4. **Use environments.** Test in development before production.
5. **Document and monitor.** Record what each function does, and watch logs and error rates.
6. **Plan for failure.** Always give the user a clear message and a retry path.

10.2 When to use what

Situation	Recommended choice
Simple validation, field updates, notifications	Deluge in Creator
Reusable simple logic across many forms	Deluge custom function
Heavy computation, libraries, OCR, PDF, custom API	Catalyst Advanced I/O function
Scheduled back-end jobs and bulk updates	Catalyst Cron function with Creator REST API
Custom interface components	Widget or embedded page

Rule of thumb: if the work needs a library, runs long, or must be tested like software, it probably belongs in Catalyst. If it is a simple rule on a form, keep it in Deluge.

241401049

11 Conclusion

Zoho Creator gives teams a fast way to build business applications, and Deluge covers most everyday needs. Its limits appear when applications become complex. Zoho Catalyst fills that gap by letting developers write Java, Node.js or Python functions that run in a serverless environment and connect to Creator over simple HTTPS calls.

The strongest design is a **hybrid**: Creator for forms, workflows and user experience, and Catalyst for heavy logic, document handling, scheduled jobs and integrations. This approach keeps the speed of low-code while adding the power and discipline of traditional development. With secured endpoints, clear JSON contracts and careful use of API limits, an organization can grow a Creator application well beyond what a visual builder alone could support.

12 References

- Zoho Catalyst Documentation, Functions overview: https://docs.catalyst.zoho.com/en/serverless/help/functions/introduction
- Zoho Catalyst Documentation, Components of Serverless: https://docs.catalyst.zoho.com/en/serverless/getting-started/components-of-serverless/
- Zoho Deluge Help, invokeUrl task: https://www.zoho.com/deluge/help/deluge-invoke-url.html
- Zoho Creator Help, scripting and API calls: https://www.zoho.com/creator/newhelp/script/get-and-post-data.html
- Zoho Catalyst product page, Functions: https://catalyst.zoho.com/functions.html

Note: Limits, availability and plan details are set by Zoho and can change. Verify them in the official documentation before building.

241401049